

Estudo Experimental de Algoritmos de Ordenacao em C: analise de comparacoes, trocas e tempo de execucao

Fabricio Rocha¹

¹Trabalho Pratico – Estruturas de Dados II

fabricio@example.com

Abstract. This paper presents a revised empirical evaluation of 13 sorting algorithms implemented in C, following the updated assignment requirements. Experiments were performed for instances with 10,000 and 1,000,000 elements under three input patterns: random, ascending and descending. For each execution, we report comparisons, swaps and run-time. The study includes the two required Quicksort variants (center pivot and median-of-three pivot), and adopts a maximum execution time of 24 hours per algorithm-instance pair, marking non-finished runs as timeout.

Resumo. Este artigo apresenta uma versao revisada da avaliacao experimental de 13 algoritmos de ordenacao implementados em C, em conformidade com o enunciado atualizado. Os algoritmos foram executados sobre instancias com 10.000 e 1.000.000 de elementos em tres padroes de entrada: aleatoria, ordenada crescente e ordenada decrescente. Em cada execucao foram coletados numero de comparacoes, numero de trocas e tempo de execucao. O estudo contempla as duas variantes exigidas de Quicksort (pivo central e mediana de tres) e adota tempo maximo de 24 horas por par algoritmo-instancia, registrando como timeout as execucoes nao finalizadas nesse limite.

1. Introducao

O objetivo deste trabalho foi comparar, de forma controlada, diferentes estrategias de ordenacao estudadas na disciplina. Foram analisados os algoritmos: bolha, bolha com parada, insercao, insercao binaria, insercao ternaria, shell, selecao, heap, quicksort com pivo central, quicksort com mediana de tres, merge, radix e bucket.

O foco da avaliacao foi observar, para diferentes perfis de entrada, como os custos teoricos se refletem na pratica em termos de comparacoes, trocas e tempo.

Este documento foi revisado para atender as alteracoes das instrucoes de confeccao do artigo, mantendo rastreabilidade entre texto, tabelas e arquivos de resultados.

2. Aderencia ao Enunciado Atualizado

O artigo atende aos pontos centrais definidos no enunciado:

- implementacao e avaliacao de 13 algoritmos de ordenacao;
- comparacao em entradas de tamanho 10.000 e 1.000.000;

- uso dos tres perfis de entrada (aleatoria, crescente e decrescente);
- apresentacao das tres metricas exigidas: comparacoes, trocas e tempo;
- inclusao das duas variantes de Quicksort exigidas (quicksortcentro e quicksortmediana);
- uso de limite de 24 horas por algoritmo/instancia para classificar timeout.

3. Metodologia Experimental

3.1. Ambiente e Instrumentacao

Os experimentos foram executados em ambiente Windows com compilacao C11. O programa experimentos.c foi utilizado para: (i) carregar as instancias, (ii) executar o algoritmo selecionado, (iii) contabilizar comparacoes e trocas, e (iv) medir tempo em segundos com `timespec_get(TIME_UTC)`.

Conforme o enunciado atualizado, foi adotado tempo maximo de 24 horas para cada par algoritmo/instancia. Execucoes nao finalizadas nesse intervalo foram registradas como timeout nos arquivos de resultados e nas tabelas deste artigo.

3.2. Instancias e Cenarios

Foram utilizadas seis instancias:

- 10.000 elementos aleatorios
- 10.000 elementos ordenados crescentemente
- 10.000 elementos ordenados decrescentemente
- 1.000.000 elementos aleatorios
- 1.000.000 elementos ordenados crescentemente
- 1.000.000 elementos ordenados decrescentemente

Total de cenarios executados: $13 \times 6 = 78$.

4. Resultados

4.1. Graficos de Tempo

A Figura 1 apresenta os tempos para $n = 10.000$. A Figura 2 apresenta os tempos para $n = 1.000.000$. Valores timeout nao sao plotados (representados como nan nos CSVs de graficos).

4.2. Graficos de Comparacoes e Trocas (**$n = 10.000$**)

As Figuras 3 e 4 destacam os custos internos de comparacoes e trocas para $n = 10.000$.

4.3. Tabelas Completas por Cenario

As tabelas a seguir apresentam os resultados completos (comparacoes, trocas e tempo) para cada tamanho e tipo de entrada.

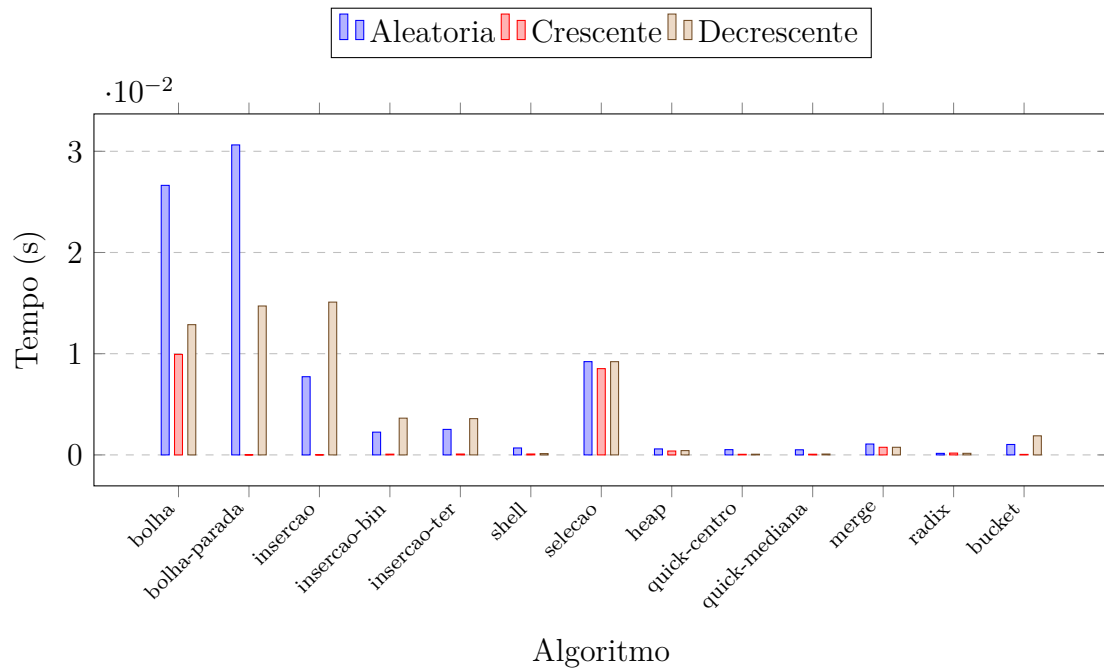


Figura 1. Tempo de execucao para $n = 10.000$.

5. Discussao

Os resultados confirmam o comportamento esperado dos algoritmos quadraticos. Em $n = 1.000.000$, bolha, selecao e insercoes gerais tornam-se inviaveis em cenarios mais custosos, com ocorrencias de timeout registradas.

Nos casos onde a entrada ja esta ordenada, algoritmos adaptativos se beneficiam: bolha com parada e insercao exibem tempos muito baixos em crescente. Por outro lado, essa vantagem nao se sustenta em perfis desfavoraveis.

Entre os algoritmos de melhor escalabilidade, radix apresentou o menor tempo na instancia aleatoria de 1.000.000 elementos (aproximadamente 0,036 s), seguido de quicksort (centro e mediana) e heap/merge.

As duas variantes de quicksort tiveram desempenho semelhante nas instancias grandes. Em entrada aleatoria, o pivo central foi ligeiramente mais rapido; em entrada decrescente, a mediana de tres foi levemente melhor em tempo. Esses dados indicam que ambas as abordagens sao competitivas, com pequenas variacoes dependentes da distribuicao.

Bucket sort mostrou forte dependencia da distribuicao dos dados: obteve bom resultado em crescente para 1.000.000, mas apresentou timeout em cenarios aleatorio e decrescente nessa mesma escala.

6. Conclusao

O estudo mostrou, de forma quantitativa, como os custos assintoticos se refletem na pratica sob diferentes entradas. Para volumes grandes, os metodos quadraticos ficam restritos a casos muito favoraveis, enquanto quicksort, heap, merge e radix mantem desempenho robusto.

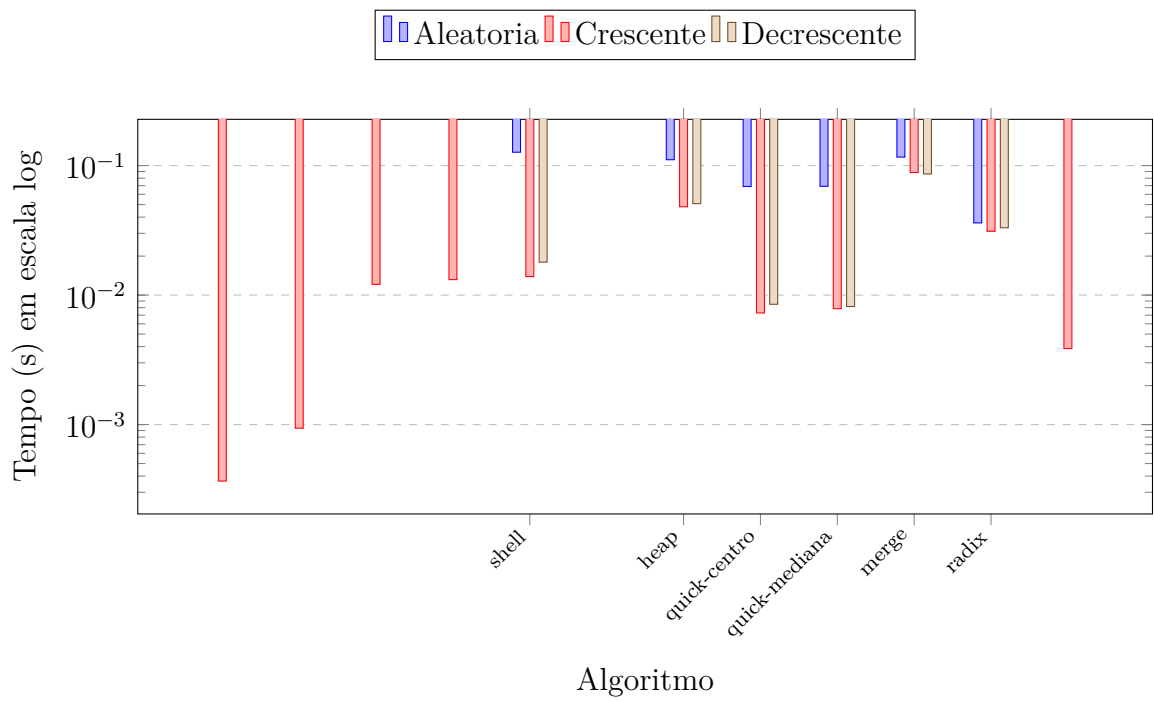


Figura 2. Tempo de execucao para $n = 1.000.000$ (escala logaritmica).

Como continuidade, recomenda-se executar multiplas repeticoes por cenario, reportar media e desvio-padrao e incluir analise estatistica para fortalecer a comparabilidade entre implementacoes e a reproducibilidade experimental.

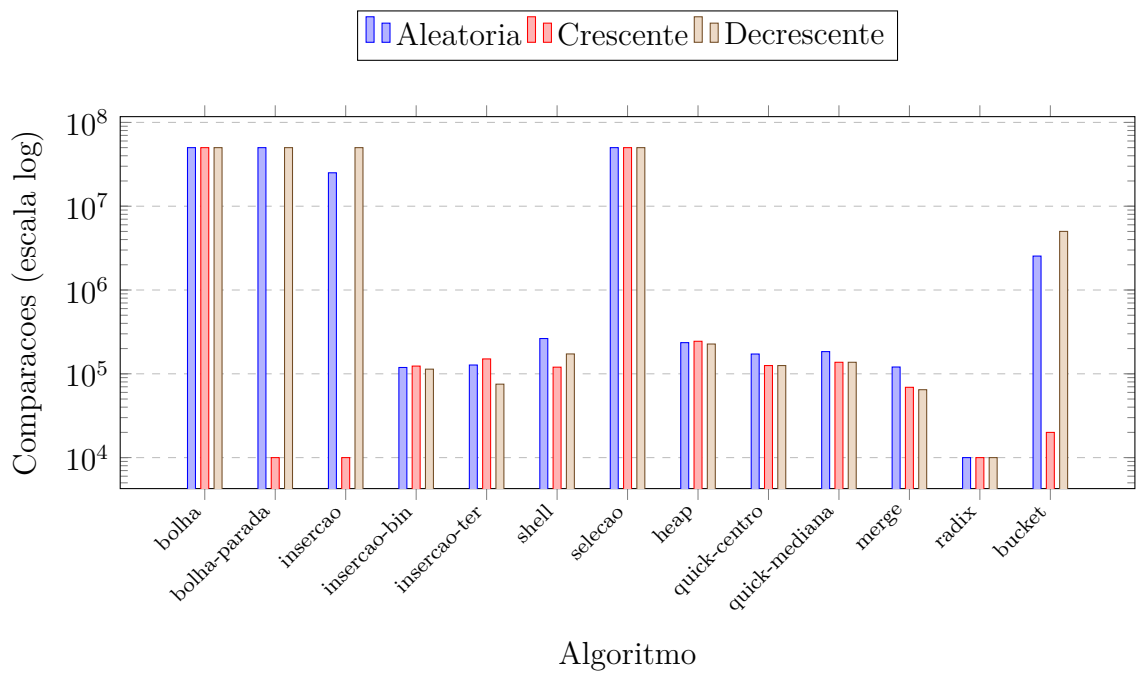


Figura 3. Numero de comparacoes para $n = 10.000$.

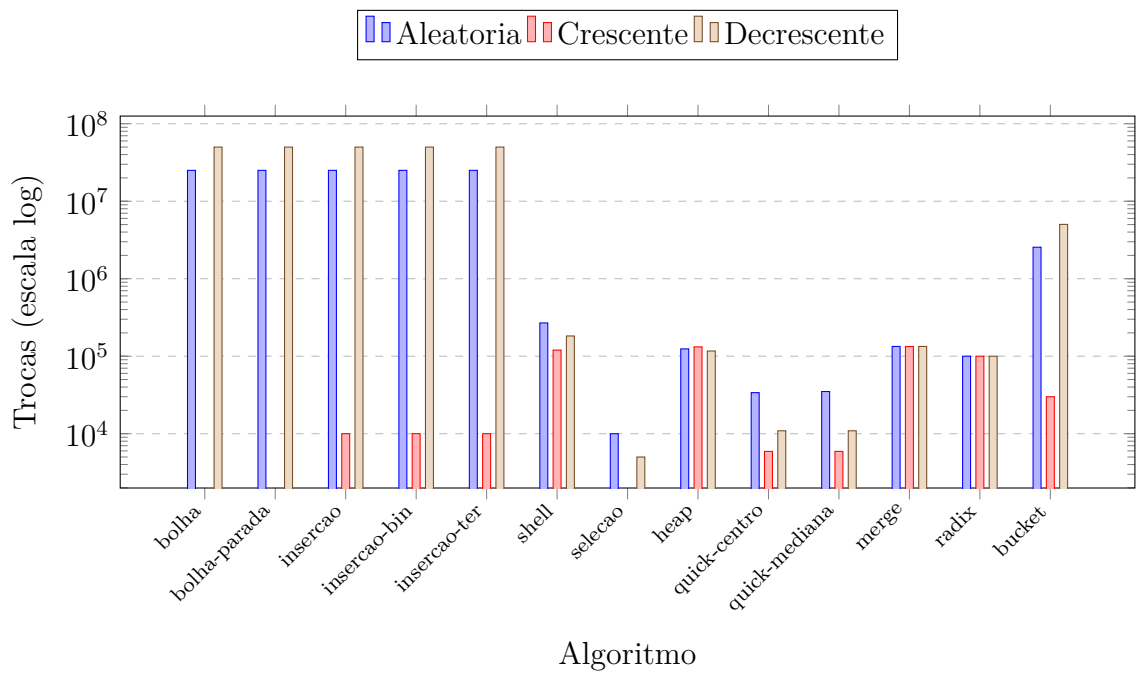


Figura 4. Numero de trocas para $n = 10.000$.

Tabela 1. Resultados para $n = 10.000$ com entrada aleatoria.

Algoritmo	Comparacoes	Trocas	Tempo (s)
bolha	49995000	25038974	0.026633024
bolha-parada	49980635	25038974	0.030628920
insercao	25048964	25048973	0.007726908
insercao-bin	118946	25048973	0.002248287
insercao-ter	127309	25048973	0.002520323
shell	263627	268749	0.000684977
selecao	49995000	9990	0.009216547
heap	235466	124254	0.000591516
quick-centro	172209	33856	0.000519037
quick-mediana	184058	35004	0.000505209
merge	120502	133616	0.001075268
radix	9999	100000	0.000152349
bucket	2540574	2550640	0.001029730

Tabela 2. Resultados para $n = 10.000$ com entrada crescente.

Algoritmo	Comparacoes	Trocas	Tempo (s)
bolha	49995000	0	0.009930372
bolha-parada	9999	0	0.000003099
insercao	9999	9999	0.000009537
insercao-bin	123617	9999	0.000074387
insercao-ter	150486	9999	0.000087023
shell	120005	120005	0.000089884
selecao	49995000	0	0.008524656
heap	244460	131956	0.000383615
quick-centro	125439	5904	0.000060081
quick-mediana	137247	5904	0.000067711
merge	69008	133616	0.000753164
radix	9999	100000	0.000179291
bucket	19989	29990	0.000050545

Tabela 3. Resultados para $n = 10.000$ com entrada decrescente.

Algoritmo	Comparacoes	Trocas	Tempo (s)
bolha	49995000	49995000	0.012869358
bolha-parada	49995000	49995000	0.014713287
insercao	49995000	50004999	0.015097141
insercao-bin	113631	50004999	0.003630400
insercao-ter	75243	50004999	0.003584385
shell	172578	182565	0.000130415
selecao	49995000	5000	0.009212971
heap	226682	116696	0.000426054
quick-centro	125452	10904	0.000068426
quick-mediana	137262	10904	0.000085592
merge	64608	133616	0.000753641
radix	9999	100000	0.000156879
bucket	5004999	5024990	0.001885891

Tabela 4. Resultados para $n = 1.000.000$ com entrada aleatoria.

Algoritmo	Comparacoes	Trocas	Tempo (s)
bolha	timeout	timeout	timeout
bolha-parada	timeout	timeout	timeout
insercao	timeout	timeout	timeout
insercao-bin	timeout	timeout	timeout
insercao-ter	timeout	timeout	timeout
shell	65724076	66226717	0.126871824
selecao	timeout	timeout	timeout
heap	36794531	19049328	0.111006498
quick-centro	27903893	4852801	0.068959951
quick-mediana	27603809	5016553	0.069190264
merge	18674269	19951424	0.116347075
radix	999999	14000000	0.036098003
bucket	timeout	timeout	timeout

Tabela 5. Resultados para $n = 1.000.000$ com entrada crescente.

Algoritmo	Comparacoes	Trocas	Tempo (s)
bolha	timeout	timeout	timeout
bolha-parada	999999	0	0.000366211
insercao	999999	999999	0.000937462
insercao-bin	18951425	999999	0.012098551
insercao-ter	23608530	999999	0.013160944
shell	18000007	18000007	0.013895988
selecao	timeout	timeout	timeout
heap	37692069	19787792	0.048102140
quick-centro	19000019	524287	0.007274866
quick-mediana	20048593	524287	0.007854223
merge	10066432	19951424	0.088405371
radix	999999	14000000	0.031106234
bucket	1999989	2999990	0.003863096

Tabela 6. Resultados para $n = 1.000.000$ com entrada decrescente.

Algoritmo	Comparacoes	Trocas	Tempo (s)
bolha	timeout	timeout	timeout
bolha-parada	timeout	timeout	timeout
insercao	timeout	timeout	timeout
insercao-bin	timeout	timeout	timeout
insercao-ter	timeout	timeout	timeout
shell	26357530	27357511	0.017968178
selecao	timeout	timeout	timeout
heap	36001436	18333408	0.050829887
quick-centro	19000036	1024286	0.008491278
quick-mediana	20048610	1024286	0.008160114
merge	9884992	19951424	0.086262941
radix	999999	14000000	0.033104897
bucket	timeout	timeout	timeout